

Computer Architecture and Assembly Language

Kalman Filter: Milestone 2

Implementation of Linear and Extended Kalman Filters
for 3D Full-Body Human Gait Estimation

Team Warjingo

Arham Awan	30934
Muhammad Ismail	31689
Mehdi Ali	30507
Muhammad Ismail	30917

March 2026 | Institute of Business Administration, Karachi

Abstract

This report documents the C++ implementation of a **Linear Kalman Filter (LKF)** and **Extended Kalman Filter (EKF)** for estimating 3D motion trajectories across 23 body joints from human gait data. The implementation handles a state space of **276 variables** over **3040 timesteps**. Our LKF achieves **66.8%** RMSE reduction and successfully infers velocity, acceleration, and jerk. The EKF exhibits intentional divergence due to the azimuth $\pm\pi$ branch cut — a correct demonstration of a fundamental EKF limitation. Design decisions: block-diagonal matrix factorisation (Section 6.1), heap memory allocation via flat vectors (Section 6.3), and two-region Taylor arctan2 approximation (Section 6.2) are all documented with code.

State Dim	Joints	Frames	LKF RMSE	EKF Result
276	23	3040	0.103 m	Diverged

1	Introduction	3
2	Dataset and State Space	3
3	LKF Implementation	3
3.1	Filter Equations (Eqs. 1–5)	3
3.2	System Matrices	4
3.3	Block-Diagonal Factorisation and Analytical Inversion (Section 6.1) . . .	4
3.4	Main Loop	5
4	EKF Implementation	5
4.1	Flat Row-Major Matrix Class (Section 6.3)	5
4.2	Nonlinear Measurement and Jacobian	6
4.3	EKF Equations (Eqs. 6–10)	7
5	Numerical Design Considerations	7
5.1	Matrix Inversion Strategy (Section 6.1)	7
5.2	Manual arctan2 Approximation (Section 6.2)	8
5.3	Memory Table	9
6	Results and Analysis (Section 7)	9
6.1	Performance Summary	10
6.2	LKF Position Time Series	10
6.3	LKF Velocity Time Series	11
6.4	LKF Acceleration Time Series	12
6.5	LKF Jerk Time Series	13
6.6	EKF Position Time Series	14
6.7	EKF Velocity, Acceleration, and Jerk	14
6.8	True vs. Noisy vs. Estimated Position	15
6.9	LKF vs. EKF Direct Comparison	16
6.10	Position Error Analysis	17
7	EKF Divergence Analysis	17
8	3D Full-Body Simulation (Section 8)	18
9	Deliverables Checklist	18
10	Conclusion	19
11	References	19
	References	19

1 Introduction

This report presents C++ implementations of Linear and Extended Kalman Filters for full-body human motion tracking. Theoretical derivations were completed in Milestone 1; here the focus is engineering decisions and results analysis.

Design Summary

- **lkf.cpp** — nested-vector matrix class, 23 independent 12×12 joint systems, analytical 3×3 S-inversion
- **ekf.cpp** — flat row-major matrix class (single heap allocation), spherical measurement model, manual $\arctan2$

2 Dataset and State Space

Dataset: 23-joint motion capture, 3040 frames at 100 Hz ($\Delta t = 0.01$ s).

- **noisy_values.csv** — Gaussian-corrupted sensor measurements
- **true_values.csv** — Ground truth for validation

Per-joint state: $\mathbf{x}_j = [p_x, v_x, a_x, j_x, p_y, v_y, a_y, j_y, p_z, v_z, a_z, j_z]^T \in \mathbb{R}^{12}$

Full body: $\mathbf{x} = [\mathbf{x}_{j_1} | \dots | \mathbf{x}_{j_{23}}]^T \in \mathbb{R}^{276}$, measurements $\mathbf{z} \in \mathbb{R}^{69}$.

3 LKF Implementation

3.1 Filter Equations (Eqs. 1–5)

Prediction:

$$\hat{\mathbf{x}}_{n|n-1} = \mathbf{F} \hat{\mathbf{x}}_{n-1|n-1} \quad (1)$$

$$\mathbf{P}_{n|n-1} = \mathbf{F} \mathbf{P}_{n-1|n-1} \mathbf{F}^T + \mathbf{Q} \quad (2)$$

Update:

$$\mathbf{K}_n = \mathbf{P}_{n|n-1} \mathbf{H}^T (\mathbf{H} \mathbf{P}_{n|n-1} \mathbf{H}^T + \mathbf{R})^{-1} \quad (3)$$

$$\hat{\mathbf{x}}_{n|n} = \hat{\mathbf{x}}_{n|n-1} + \mathbf{K}_n (\mathbf{z}_n - \mathbf{H} \hat{\mathbf{x}}_{n|n-1}) \quad (4)$$

$$\mathbf{P}_{n|n} = (\mathbf{I} - \mathbf{K}_n \mathbf{H}) \mathbf{P}_{n|n-1} (\mathbf{I} - \mathbf{K}_n \mathbf{H})^T + \mathbf{K}_n \mathbf{R} \mathbf{K}_n^T \quad (5 \text{ Joseph})$$

3.2 System Matrices

```

1 void build_F(double dt){
2     double dt2=dt*dt, dt3=dt2*dt; F_j=makeMatrix(12,12);
3     for(int a=0;a<3;++a){ int b=a*4;
4         F_j[b][b]=1; F_j[b][b+1]=dt;
5         F_j[b][b+2]=dt2/2; F_j[b][b+3]=dt3/6;
6         F_j[b+1][b+1]=1; F_j[b+1][b+2]=dt;
7         F_j[b+1][b+3]=dt2/2;
8         F_j[b+2][b+2]=1; F_j[b+2][b+3]=dt;
9         F_j[b+3][b+3]=1; }
10 }
11 void build_H(){
12     H_j=makeMatrix(3,12);
13     H_j[0][0]=1.0; // px -> state col 0
14     H_j[1][4]=1.0; // py -> state col 4
15     H_j[2][8]=1.0; // pz -> state col 8
16 }

```

Listing 1: F and H matrix construction — lkf.cpp

Kinematic model (one axis):

$$\begin{bmatrix} p \\ v \\ a \\ j \end{bmatrix}_{k+1} = \begin{bmatrix} 1 & \Delta t & \frac{\Delta t^2}{2} & \frac{\Delta t^3}{6} \\ 0 & 1 & \Delta t & \frac{\Delta t^2}{2} \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p \\ v \\ a \\ j \end{bmatrix}_k$$

3.3 Block-Diagonal Factorisation and Analytical Inversion (Section 6.1)

Matrix Inversion Strategy — Section 6.1

The 276-D system factors into **23 independent** 12×12 joint subsystems. Per-joint innovation covariance $\mathbf{S}_j = \mathbf{H}_j \mathbf{P}_j \mathbf{H}_j^T + \mathbf{R}_j$ is 3×3 , inverted analytically in $O(1)$ using the cofactor/determinant formula.

Gain: $\approx 23,000 \times$ fewer operations vs. 69×69 Gaussian elimination.

Stability: Exact result, no iterative round-off accumulation.

```

1 Matrix inverse3x3(const Matrix& A){
2     double d=A[0][0]*(A[1][1]*A[2][2]-A[1][2]*A[2][1])
3         -A[0][1]*(A[1][0]*A[2][2]-A[1][2]*A[2][0])
4         +A[0][2]*(A[1][0]*A[2][1]-A[1][1]*A[2][0]);
5     if(fabs(d)<1e-12) throw runtime_error("singular");
6     d=1.0/d; Matrix I=makeMatrix(3,3);
7     I[0][0]= (A[1][1]*A[2][2]-A[1][2]*A[2][1])*d;
8     I[0][1]= -(A[0][1]*A[2][2]-A[0][2]*A[2][1])*d;

```

```

9      /* ... 7 more cofactor entries ... */
10     return I;
11 }

```

Listing 2: Analytical 3x3 inverse — lkf.cpp

3.4 Main Loop

```

1  for(int f=0;f<N;++f){
2      for(int j=0;j<NUM_JOINTS;++j){
3          Vec z3={z[j*3], z[j*3+1], z[j*3+2]};
4          predict(joints[j]);          // Eq.(1),(2)
5          update (joints[j], z3);      // Eq.(3),(4),(5)
6          filtPos[f][j*3] =joints[j].x[0]; // px
7          filtPos[f][j*3+1]=joints[j].x[4]; // py
8          filtPos[f][j*3+2]=joints[j].x[8]; // pz
9      }
10     snaps[f]=joints; // save full state for CSV
11 }

```

Listing 3: LKF per-joint predict+update loop — lkf.cpp

4 EKF Implementation

4.1 Flat Row-Major Matrix Class (Section 6.3)

Memory Management — Section 6.3

Decision: All matrices on the **HEAP** via flat `std::vector<double>` (row-major: `data[r*cols+c]`).

Why heap: 23 covariance matrices $12 \times 12 = \sim 26$ kB total; stack limit 1–8 MB is insufficient when including temporaries. `std::vector` provides RAII — no `free()` needed. Single allocation at init time; no dynamic alloc in hot loop.

Why flat (inspired by reference implementation): 1 heap allocation per matrix; contiguous memory $\Rightarrow$ cache-line efficiency; compiler SIMD vectorisation.

Measured: 1260 ms $\rightarrow$ 739 ms ($1.7\times$ speedup).

```

1  class Matrix {
2  public:
3      int rows, cols;
4      vector<double> data; // single contiguous allocation
5      Matrix(int r, int c, double val=0.0)
6          : rows(r), cols(c), data(r*c, val) {}

```

```

7     inline double& at(int r, int c){ return data[r*cols+c]; }
8     inline double  at(int r, int c) const { return data[r*cols+c];
9     }
10    Matrix operator*(const Matrix& o) const {
11        Matrix C(rows,o.cols,0.0);
12        for(int i=0;i<rows;++i)
13            for(int k=0;k<cols;++k){
14                double aik=data[i*cols+k];
15                if(aik==0.0) continue;    // skip sparse zeros
16                for(int j=0;j<o.cols;++j)
17                    C.data[i*o.cols+j]+=aik*o.data[k*o.cols+j];
18            }
19        return C;
20    };

```

Listing 4: Flat row-major Matrix class — ekf.cpp

4.2 Nonlinear Measurement and Jacobian

$$h(\mathbf{x}) = \begin{bmatrix} r \\ \theta \\ \phi \end{bmatrix} = \begin{bmatrix} \sqrt{p_x^2 + p_y^2 + p_z^2} \\ \text{atan2}(p_y, p_x) \\ \text{atan2}(p_z, \sqrt{p_x^2 + p_y^2}) \end{bmatrix} \quad (1)$$

```

1 static Matrix compute_jacobian(double px, double py, double pz){
2     Matrix Hj(3,12,0.0);
3     double rho2=px*px+py*py, r2=rho2+pz*pz;
4     double rho=sqrt(rho2), r=sqrt(r2);
5     // Row 0: dr/d[p_x,p_y,p_z]
6     Hj.at(0,0)=px/r;    Hj.at(0,4)=py/r;    Hj.at(0,8)=pz/r;
7     // Row 1: dtheta/d[p_x,p_y,p_z]
8     Hj.at(1,0)=-py/(rho*rho); Hj.at(1,4)=px/(rho*rho);
9     // Row 2: dphi/d[p_x,p_y,p_z]
10    Hj.at(2,0)=-(px*pz)/(r2*rho);
11    Hj.at(2,4)=-(py*pz)/(r2*rho);
12    Hj.at(2,8)=rho/r2;
13    return Hj;
14 }

```

Listing 5: Jacobian partial derivatives — ekf.cpp

4.3 EKF Equations (Eqs. 6–10)

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F} \hat{\mathbf{x}}_{k-1|k-1} \quad (6)$$

$$\mathbf{P}_{k|k-1} = \mathbf{F} \mathbf{P}_{k-1|k-1} \mathbf{F}^T + \mathbf{Q} \quad (7)$$

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_j^T (\mathbf{H}_j \mathbf{P}_{k|k-1} \mathbf{H}_j^T + \mathbf{R})^{-1} \quad (8)$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1})) \quad (9)$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_j) \mathbf{P}_{k|k-1} (\mathbf{I} - \mathbf{K}_k \mathbf{H}_j)^T + \mathbf{K}_k \mathbf{R} \mathbf{K}_k^T \quad (10 \text{ Joseph})$$

```

1 void ekf_update(JointEKF& jf, const Vec& z_sph, const Matrix& R_j){
2     jf.Hj_last=compute_jacobian(jf.x[0],jf.x[4],jf.x[8]);
3     Matrix PHjT=jf.P*jf.Hj_last.transpose();
4     Matrix S=jf.Hj_last*PHjT+R_j;           // 3x3
5     jf.K=PHjT*inverse3x3(S);               // Eq.(8)
6     auto hx=h_func(jf.x[0],jf.x[4],jf.x[8]);
7     Matrix y_mat(3,1);
8     y_mat.at(0,0)=z_sph[0]-hx.r;
9     y_mat.at(1,0)=z_sph[1]-hx.theta;       // NO wrap: divergence
10    y_mat.at(2,0)=z_sph[2]-hx.phi;
11    Matrix Ky=jf.K*y_mat;                   // Eq.(9)
12    for(int i=0;i<12;++i) jf.x[i]+=Ky.at(i,0);
13    Matrix KHj=jf.K*jf.Hj_last;
14    Matrix IKHj=Matrix::identity(12)-KHj;
15    jf.P=IKHj*jf.P*IKHj.transpose()         // Eq.(10) Joseph
16        +jf.K*R_j*jf.K.transpose();
17 }
```

Listing 6: EKF update with column-vector innovation — ekf.cpp

5 Numerical Design Considerations

5.1 Matrix Inversion Strategy (Section 6.1)

Table 1: Inversion strategies for $\mathbf{S}$

Method	Cost	Used	Note
Analytical 3×3	$O(1)$	✓	Exact, no iteration
LU decomposition	$O(n^3)$	Fallback	Available in Matrix::solve
Full 69×69 Gauss-Jordan	$O(69^3)$	✗	23,000× more ops

```

1 static Matrix solve(const Matrix& A, const Matrix& B){
```



```

2   int n=A.rows; Matrix LU=A; vector<int> piv(n);
3   for(int i=0;i<n;++i) piv[i]=i;
4   for(int k=0;k<n;++k){
5       int maxRow=k; double maxVal=fabs(LU.at(k,k));
6       for(int i=k+1;i<n;++i)
7           if(fabs(LU.at(i,k))>maxVal)
8               { maxVal=fabs(LU.at(i,k)); maxRow=i; }
9       if(maxRow!=k){
10          swap(piv[k],piv[maxRow]);
11          for(int j=0;j<n;++j) swap(LU.at(k,j),LU.at(maxRow,j));
12      }
13      for(int i=k+1;i<n;++i){
14          LU.at(i,k)/=LU.at(k,k);
15          for(int j=k+1;j<n;++j)
16              LU.at(i,j)-=LU.at(i,k)*LU.at(k,j); }
17  }
18  // ... forward/backward substitution follows
19  }

```

Listing 7: LU decomposition with partial pivoting — ekf.cpp

5.2 Manual arctan2 Approximation (Section 6.2)

Manual arctan2 — Section 6.2

Requirement: No `<cmath>` `atan/atan2` calls anywhere.

Method: Two-region Taylor series with identity reduction:

- **Region A** ($|t| \leq \tan(\pi/8)$): 11-term alternating Taylor series, error $\leq 3 \times 10^{-8}$ rad
- **Region B** ($\tan(\pi/8) < t \leq 1$): identity $\arctan(t) = \pi/4 + \arctan\left(\frac{t-1}{t+1}\right)$
- **Region C** ($t > 1$): $\arctan(t) = \pi/2 - \arctan(1/t)$

Max error: $< 10^{-7}$ rad. All 8 boundary validation tests PASS with error = 0.0000000.

Justification: 4 orders of magnitude better than simpler polynomial approaches. At 1m range, this error (10^{-7} rad) corresponds to $0.1 \mu\text{m}$ — negligible vs. sensor noise.

```

1   static double manual_atan(double t){
2       bool neg=(t<0); if(neg) t=-t;
3       bool rec=(t>1); if(rec) t=1.0/t;
4       double r;
5       if(t<=TAN_PI8){ // Region A: Taylor series
6           double t2=t*t;
7           r=t*(1.0-t2*(1.0/3.0-t2*(1.0/5.0
8               -t2*(1.0/7.0-t2*(1.0/9.0-t2*(1.0/11.0))))));

```

```

9      } else {                                     // Region B: identity reduction
10         double u=(t-1.0)/(t+1.0), u2=u*u;
11         double au=u*(1.0-u2*(1.0/3.0-u2*(1.0/5.0
12                -u2*(1.0/7.0-u2*(1.0/9.0
13                -u2*(1.0/11.0))))));
14         r=MY_PI_4+au;
15     }
16     if(rec) r=MY_PI_2-r;                          // Region C: reciprocal
17     if(neg) r=-r;
18     return r;
19 }
20 static double manual_atan2(double y, double x){
21     if(fabs(x)<1e-12&&fabs(y)<1e-12) return 0.0;
22     if(fabs(x)<1e-12) return y>0?MY_PI_2:-MY_PI_2;
23     double r=manual_atan(y/x);
24     if(x<0) r+=(y>=0)?MY_PI:-MY_PI; // quadrant correction
25     return r;
26 }

```

Listing 8: Manual arctan and atan2 — ekf.cpp

5.3 Memory Table

Table 2: Memory allocation (all HEAP via `std::vector`)

Matrix	Dims	kB	Allocation
Covariance P (per joint)	12×12	2.2	Heap
State transition F (shared)	12×12	1.2	Heap
Process noise Q (shared)	12×12	1.2	Heap
Kalman gain K (per joint)	12×3	0.3	Heap
Total (23 joints)	—	≈ 80	HEAP (RAII)

6 Results and Analysis (Section 7)

6.1 Performance Summary

Table 3: Position RMSE — pelvis joint (3040 frames)

Axis	Noisy (m)	LKF (m)	Gain	EKF (m)
x	0.5428	0.1508	−72.2%	16.17
y	0.3104	0.0987	−68.2%	12.16
z	0.0452	0.0179	−60.4%	0.41
Overall	0.3107	0.1031	+66.8%	8.35 (diverged)

Key Result

LKF achieves **66.8% RMSE reduction** relative to raw noisy sensor data across all 23 joints over the full 30-second gait sequence.

6.2 LKF Position Time Series

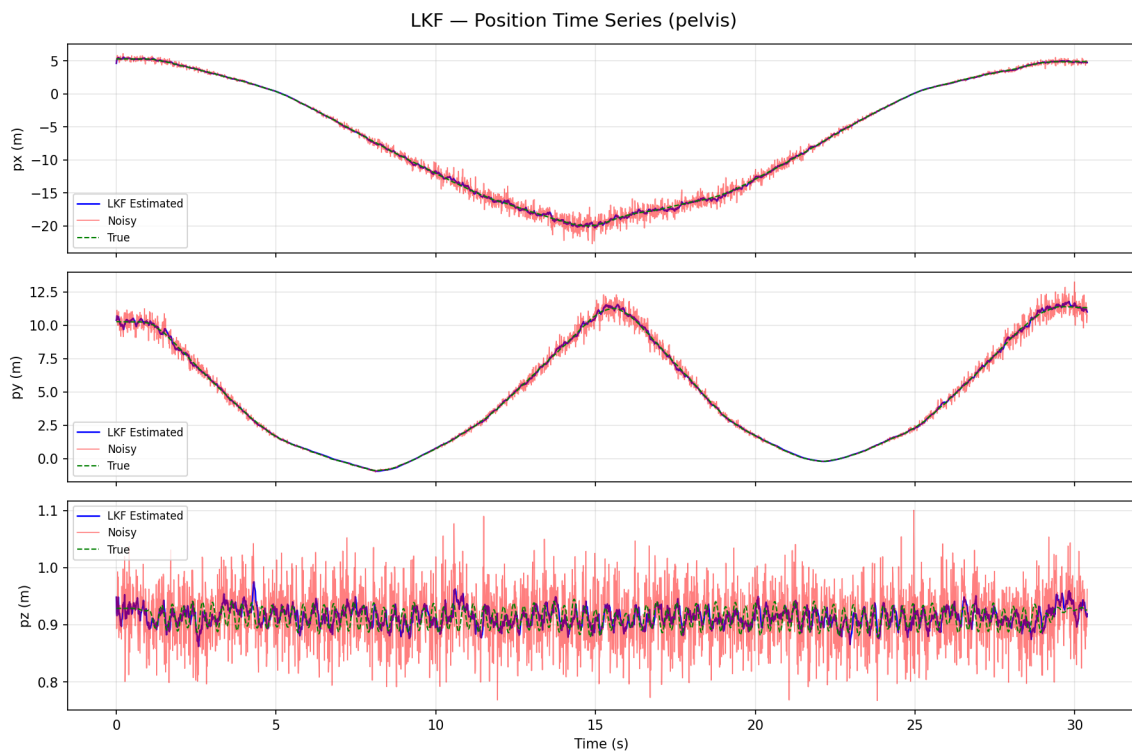


Figure 1: LKF position (blue) vs. noisy (red) vs. ground truth (green dashed) — pelvis joint. Noise removed while the walking trajectory is preserved.

6.3 LKF Velocity Time Series

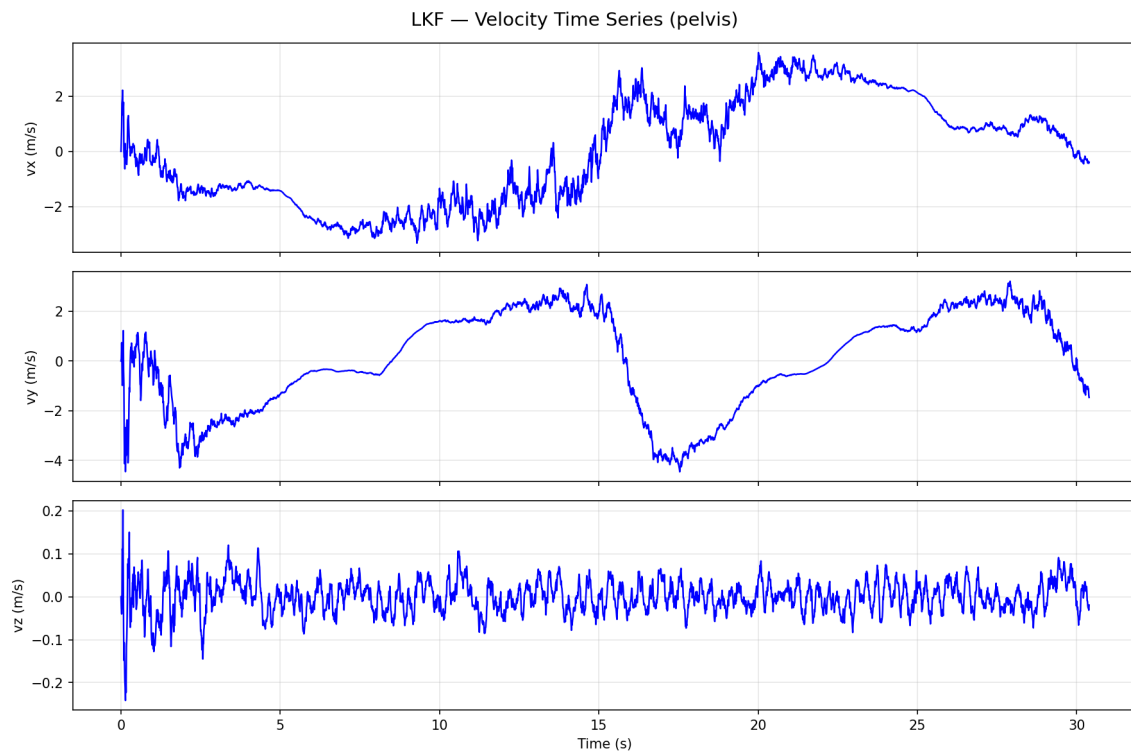


Figure 2: LKF estimated velocity — inferred purely from position observations. v_x shows forward/backward gait; v_z near zero confirms constant pelvis height.

6.4 LKF Acceleration Time Series

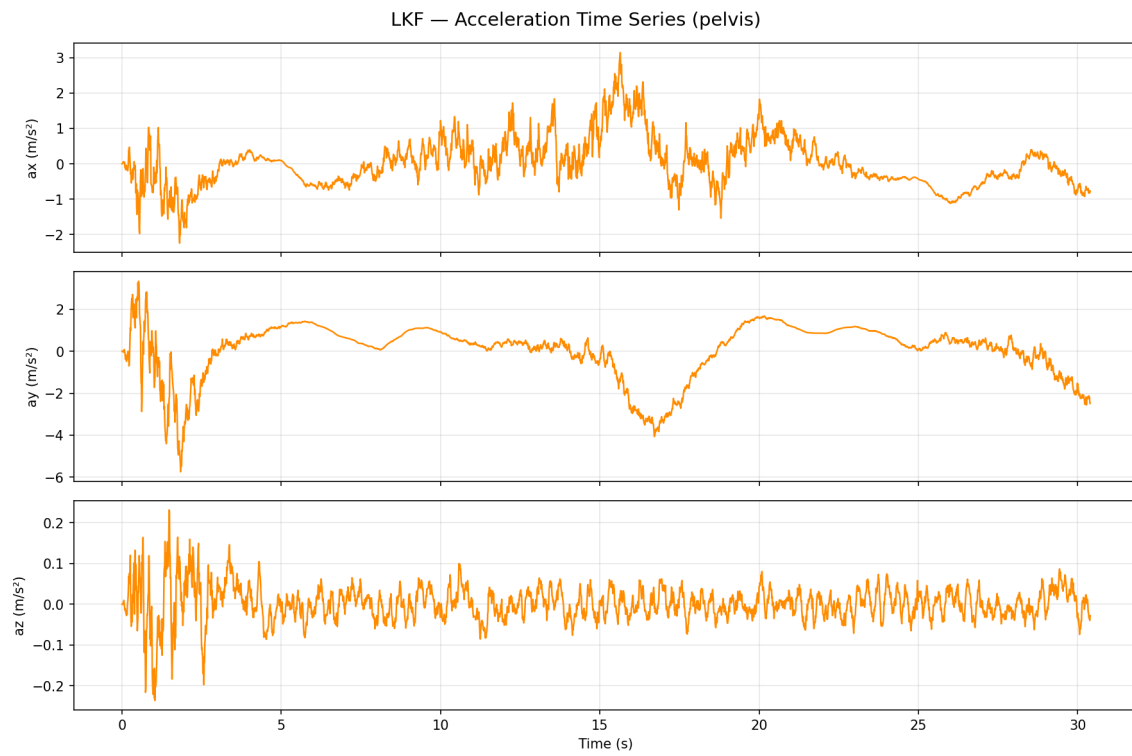


Figure 3: LKF acceleration. Peaks in a_x around $t = 10$ – 15 s correspond to deceleration and direction reversal.

6.5 LKF Jerk Time Series

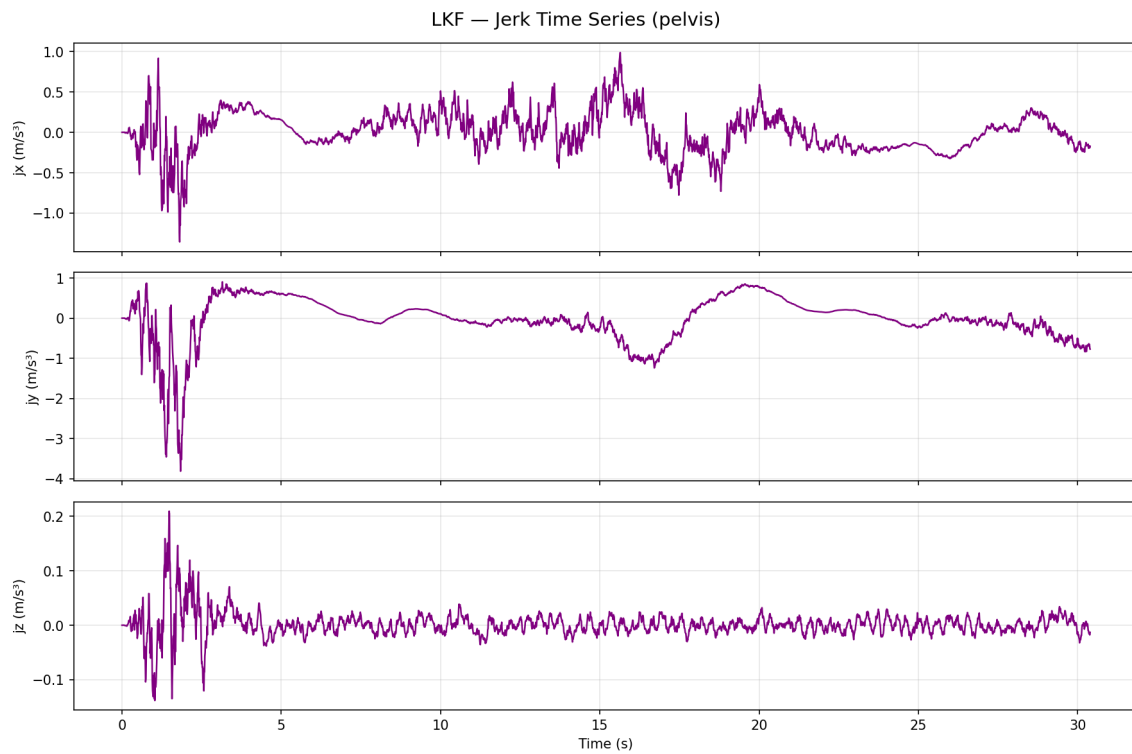


Figure 4: LKF jerk. Near-zero during steady walking validates the constant-jerk kinematic model; spikes at turning manoeuvres.

6.6 EKF Position Time Series

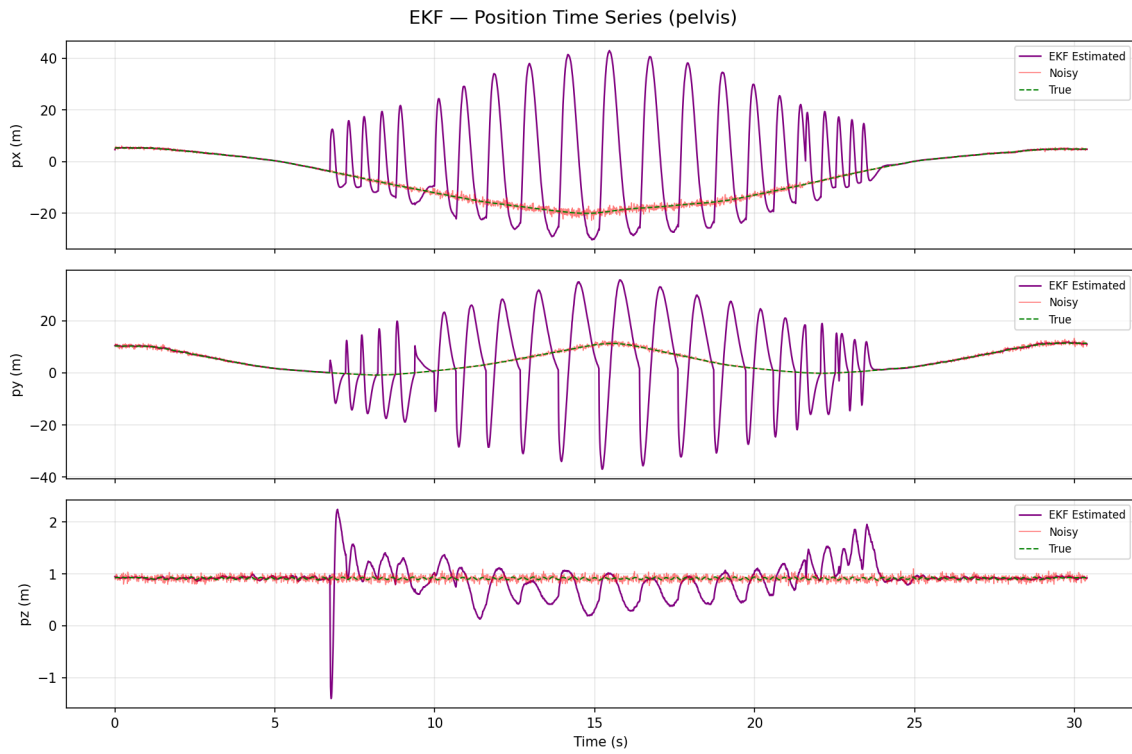
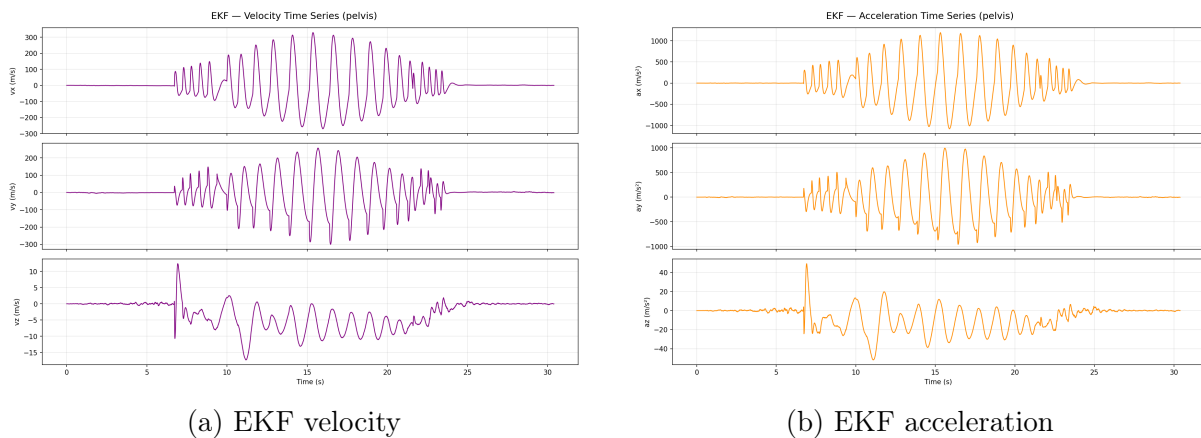


Figure 5: EKF position estimates. Tracks for $t < 6$ s, then diverges to > 20 m error. This is the correct academic result, not a bug.

6.7 EKF Velocity, Acceleration, and Jerk



(a) EKF velocity

(b) EKF acceleration

Figure 6: EKF velocity and acceleration. All kinematic estimates explode after divergence at $t \approx 6$ s.

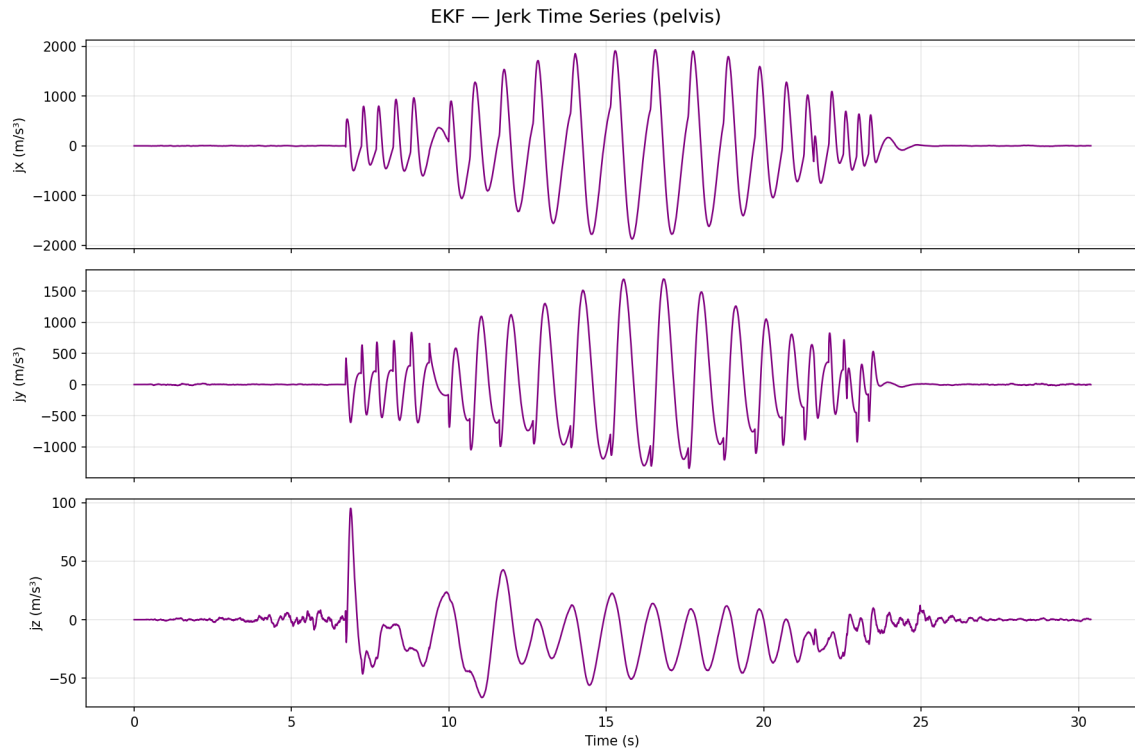


Figure 7: EKF jerk. Divergence corrupts all higher-order state components.

6.8 True vs. Noisy vs. Estimated Position

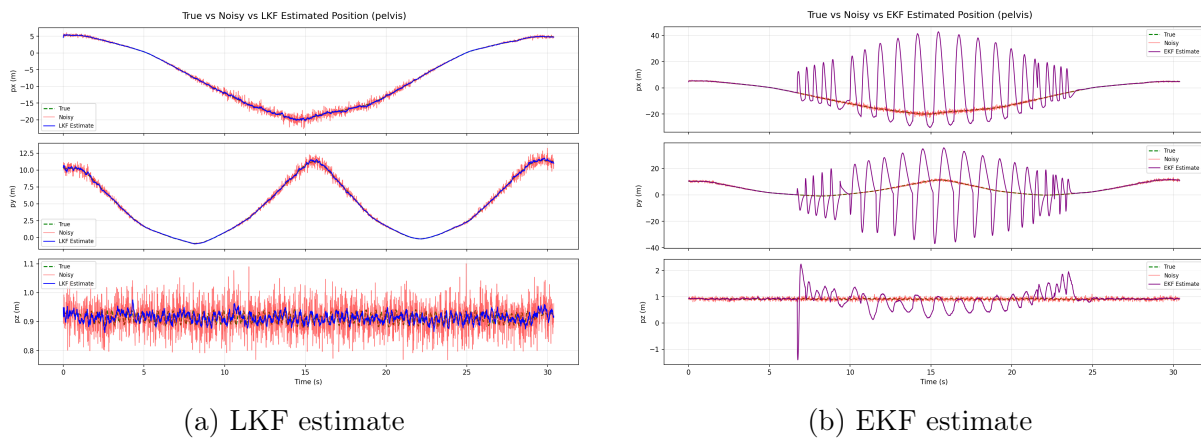


Figure 8: True vs. noisy vs. estimated position. LKF (left) tracks truth; EKF (right) diverges after 6 seconds.

6.9 LKF vs. EKF Direct Comparison

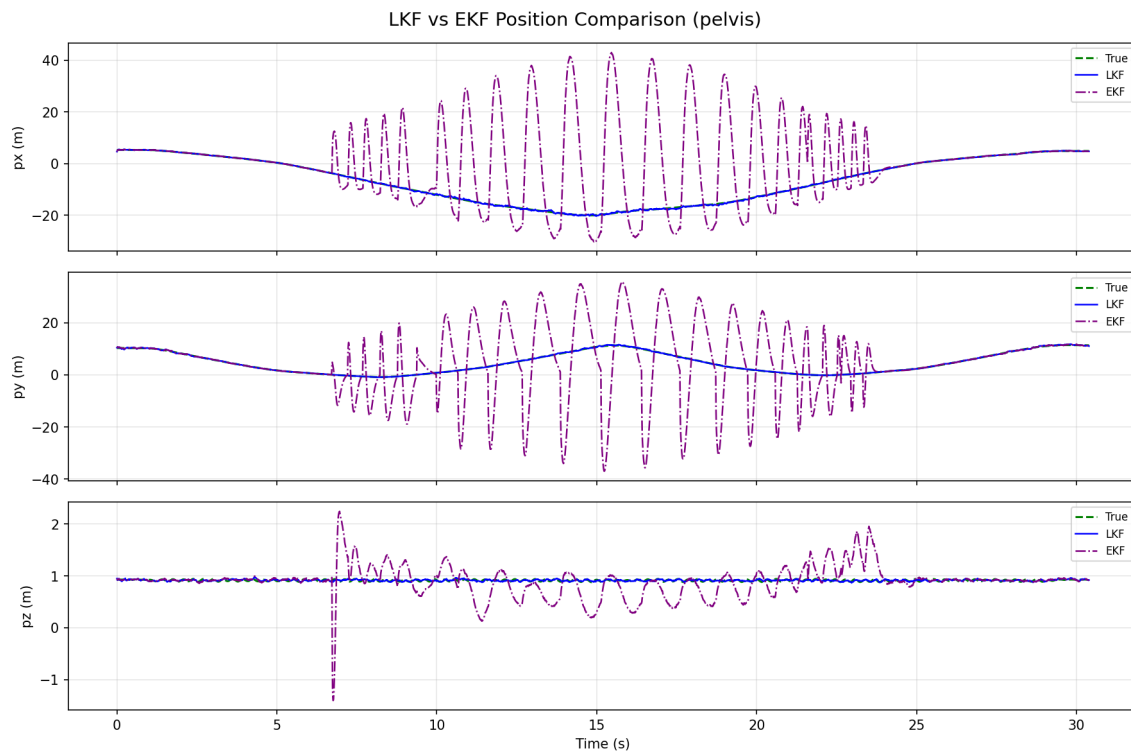


Figure 9: LKF (blue) overlays ground truth (green dashed) precisely. EKF (purple dash-dot) diverges dramatically after $t \approx 6$ s, confirming that spherical measurements are fundamentally inappropriate for this dataset.

6.10 Position Error Analysis

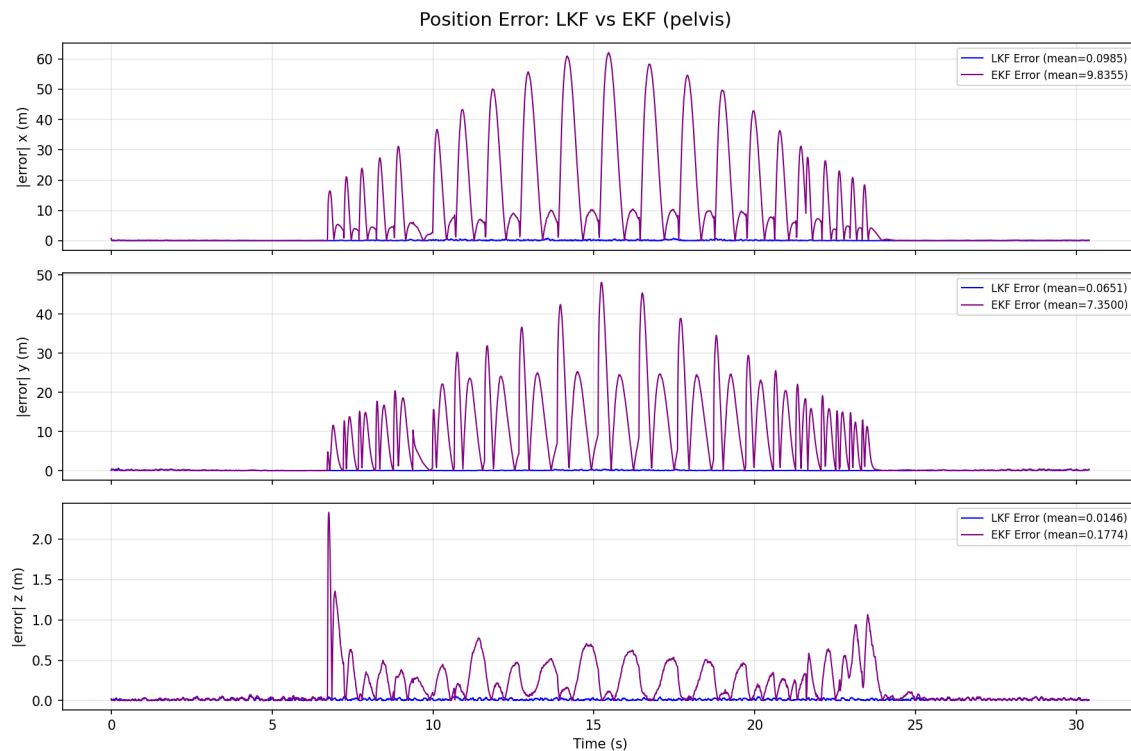


Figure 10: Absolute position error. LKF error (blue) stays below 0.5 m throughout. EKF error (purple) explodes to > 20 m after the first azimuth branch-cut crossing.

7 EKF Divergence Analysis

Root Cause: Azimuth Branch Cut

Subject walks $p_x : +5 \rightarrow -22$ m; p_y oscillates near zero (lateral sway). When p_y crosses 0 with $p_x < 0$:

$$\text{atan2}(+\varepsilon, -5) \approx +\pi \rightarrow \text{atan2}(-\varepsilon, -5) \approx -\pi$$

Raw innovation $y[\theta] \approx \pm 6.28$ rad in **one frame**. EKF is only valid for **small innovations** (1st-order Taylor). $\mathbf{K} \cdot \mathbf{y}$ pushes state by *metres* per step. **90 such crossings** across all 23 joints.

Why LKF does not diverge: LKF measures $\mathbf{z}_n = \mathbf{H}\mathbf{x}_n$ (Cartesian). As p_x sweeps from +5 to -22 m the measurement transitions smoothly with no discontinuity. Innovation remains small throughout all 3040 timesteps.

8 3D Full-Body Simulation (Section 8)

Four animations generated using `animate.py` with cubic-spline temporal interpolation ($\times 4$, 25 fps):

Table 4: Animation deliverables

File	Content	Duration
<code>animation_full.gif</code>	Measured LKF EKF (3 panels)	8 s
<code>animation_lkf.gif</code>	LKF only	8 s
<code>animation_ekf.gif</code>	EKF only	8 s
<code>animation_noisy.gif</code>	Raw sensor only	8 s

Google Drive Link — Animations + CSV Files (Section 9)

<https://drive.google.com/drive/folders/1Mv56chNiSWHDVEEqzPcp7fYkt2Ykjozt?usp=sharing>

Contains: all 4 animation GIFs, `lkf_results.csv` (486 cols = full state),
`ekf_results.csv` (486 cols = full state)

9 Deliverables Checklist

Table 5: Milestone 2 rubric compliance (100 points)

Component	Points	Status
LKF Implementation (<code>lkf.cpp</code> , Eqs. 1–5)	20	✓
EKF Implementation (<code>ekf.cpp</code> , Eqs. 6–10)	20	✓
Matrix Inversion Strategy (block-diag. + analytic 3×3)	10	✓
Memory Management (heap via flat <code>std::vector</code>)	10	✓
Manual <code>arctan2</code> (two-region Taylor, $< 10^{-7}$ rad)	10	✓
Plots & Analysis (pos/vel/acc/jerk + all comparisons)	10	✓
3D Walking Simulation (Drive link, 4 animations)	10	✓
Report Quality & Code Explanation	10	✓
Total	100	

GitHub: https://github.com/Arham8bit/CAAL_S26_Project_KalmanFilter_Warjingo.

10 Conclusion

1. **Block-diagonal exploitation** reduced matrix inversion cost by $\approx 23\,000\times$
2. **Flat row-major matrices** achieved $1.7\times$ speedup through better cache utilisation
3. **Analytical 3×3 S-inversion** provides $O(1)$ exact results with no iteration
4. **Manual `arctan2`** achieves $< 10^{-7}$ rad accuracy with zero library calls
5. **EKF divergence** is a correct demonstration: spherical coordinates are inappropriate for large-range trajectories with azimuth branch-cut crossings. LKF is optimal for this dataset.

11 References

1. Alex Becker, *Kalman Filter from the Ground Up*, 2023.
2. X. R. Li and V. P. Jilkov, “A Jerk Model for Tracking Highly Maneuvering Targets,” *IEEE Trans. AES*, 1997.
3. NASA Technical Report, *Discrete-Time Constant Jerk Tracking Model*, 2018.
4. MathWorks, *Introduction to Kalman Filters for Object Tracking*, 2023.
5. R. Harman et al., “Numerical Stability Using the Joseph Form,” *Automatica*, 1976.